



School of Computer Science & Technologies

Field of study: Computer Science

Oğuzhan Türksoy

Index number 55032

**ShoeStore — Full Retail Platform: Web Application for Unified Shoe
Store Management and E-Commerce**

Bachelor thesis written under supervision of

dr Krzysztof Rychlicki-Kicior

Warsaw, 2026

1. Basic Information

Project title	ShoeStore — Full Retail Platform: Web Application for Unified Shoe Store Management and E-Commerce.
Project goal	The goal of the project is to design and deliver a single, production-ready software platform that simultaneously serves the operational needs of a physical shoe store and its online sales channel. The system addresses a real problem faced by small-to-medium shoe retailers: the in-store inventory, the e-commerce catalogue, and the staff workflow normally live in different tools, which produces duplicated data entry, stale stock counts, and overselling. ShoeStore solves this by running one Spring Boot backend and one PostgreSQL database behind three role-specific React frontends — an Admin Panel for the store owner, a mobile Staff PWA for in-store employees, and a public Storefront for online customers. Because all three apps read and write the same product, inventory, and order tables, the stock count visible to a customer at checkout is always the same number that the staff sees when scanning a shoe in the store, with no synchronisation step in between.
Short project description	ShoeStore is a full-stack web application made up of one Spring Boot 3 REST API, three independent React 18 single-page applications, and an nginx reverse proxy that fans the public traffic out to the right container. All four services and a PostgreSQL 15 database are orchestrated with a single docker compose file, so the whole platform comes up with one command on a developer laptop. The Admin Panel (served at /admin) lets the owner manage the product catalogue, configure per-size stock, set discounts, view online orders, monitor staff sales, generate AI-assisted product photographs, and read a 30-day revenue dashboard. The Staff PWA (/app) is installable on a phone home-screen; staff scan the QR code on every shoe to sell it, accept a return, or receive new stock — each action writes a row into a single stock_movements audit table. The Storefront (/store) is the public e-commerce site. Customers browse the catalogue, add shoes to a cart, and check out anonymously with Stripe PaymentIntents. Stock is reserved with a pessimistic row lock at the moment a PaymentIntent is created, so two customers cannot pay for the last pair in stock; Stripe webhook traffic finalises or releases the reservation idempotently.
Competition analysis	Shopify is the most widely used commercial SaaS for online shoe retail. Its strengths are a polished checkout, a large theme/app marketplace, and zero infrastructure overhead. The disadvantage for the use case targeted here is that Shopify is built around the online channel — in-store sales need a separate POS extension and additional hardware, the source code is closed, and per-store fees scale with revenue. WooCommerce is an open-source WordPress plugin. It gives the retailer full code ownership, but provides no in-store staff tooling, depends on WordPress upgrade cycles, and shifts the entire operational burden (plugins, hosting, security patches) onto the store. Lightspeed Retail is a closed SaaS aimed at physical stores. Its POS and inventory modules are mature, but the e-commerce add-on is less integrated than a purpose-built storefront and carries per-seat pricing. ShoeStore differentiates itself by serving online and offline channels from one backend and one PostgreSQL database. Because both channels read the same inventory row (with a pessimistic write lock at checkout), inventory consistency is a property of the schema rather than of a synchronisation job. The whole stack is open-source and self-hostable; the only paid dependency is Stripe (per-transaction).
List of technologies	- Spring Boot 3.x (Java 17, Maven) - Spring Security 6 with stateless JWT in an HttpOnly cookie - Spring Data JPA / Hibernate - Flyway database migrations (V1–V20)

	- PostgreSQL 15 - Stripe Java SDK — PaymentIntents + Webhooks - ZXing (server-side QR code rendering) - Jakarta Mail / Spring Mail over an SMTP relay (Brevo) for the order-confirmation email - Google Gemini 2.5 Flash Image API (AI product photography) - React 18 + Vite (admin-web, staff-pwa, storefront-web) - React Router v6, Axios, html5-qrcode, @stripe/stripe-js, @stripe/react-stripe-js - nginx (reverse proxy, rate limiting, CSP, TLS termination point) - Docker + Docker Compose (one-command deployment)
Description of technologies used in the project and the justification	The backend is written in Java 17 on Spring Boot 3 because Spring Boot offers a mature, production-tested REST stack with very little boilerplate, and because Spring Security's filter chain composes well with the JWT-in-HttpOnly-cookie design used here. JPA with Hibernate maps the domain entities (Product, ProductSize, CustomerOrder, StockMovement, ...) to PostgreSQL tables, while Flyway owns every schema change as a numbered SQL migration so the database can be rebuilt deterministically. PostgreSQL 15 was selected for its mature transactional guarantees and excellent JPA compatibility. The stock-reservation logic at checkout relies on PostgreSQL's row-level pessimistic locks; the analytics aggregation that powers the Admin Dashboard uses PostgreSQL's date_trunc and grouping operators directly. All three frontends are built with React 18 and Vite. React was chosen because its component model maps cleanly to the user flows (catalogue grid, scanner, dashboard), and because using the same library across three apps allows utility code and conventions to be shared. Vite was selected over Create React App because of substantially faster HMR and a smaller production bundle. Stripe handles payments. PaymentIntents combined with Stripe Elements let the storefront accept cards without the storefront container ever seeing a PAN; the webhook (signed with HMAC) gives a reliable secondary confirmation path independent of the browser. nginx is the single public entry point. It terminates the network boundary, applies rate limits per endpoint (login, checkout, generic API), enforces a strict Content-Security-Policy, and routes by URL prefix (/admin, /app, /store, /api) to the right container — which side-steps CORS because every browser request comes from the same origin. Docker Compose stitches the database, backend, three frontends, and the proxy together into one declarative file so the platform starts with `docker compose up`.

2. Key Issues Relating to Project Implementation

2.1 Single-origin Gateway with nginx

The platform exposes a single public origin through an nginx reverse proxy. The three React SPAs and the `/api/*` surface of the Spring Boot backend are served behind one port, with routing decided by URL prefix. Running everything under one origin means the `HttpOnly` auth cookie is naturally scoped to the whole site, browser CORS does not apply to the SPA-to-API hop, and rate limits and security headers are enforced in one place. The Stripe-webhook location is declared before the generic `/api/` block so it wins the match, and `proxy_request_buffering` is disabled there because Stripe verifies its HMAC signature over the exact byte stream of the body.

```
# reverse-proxy/nginx.conf
limit_req_zone $binary_remote_addr zone=login_zone:10m rate=5r/m;
limit_req_zone $binary_remote_addr zone=checkout_zone:10m rate=10r/m;
limit_req_zone $binary_remote_addr zone=api_zone:10m rate=60r/s;

location = /api/storefront/checkout/webhook {
    proxy_request_buffering off;
    proxy_pass http://backend:8080/api/storefront/checkout/webhook;
}

location = /api/auth/login {
    limit_req zone=login_zone burst=3 nodelay;
    proxy_pass http://backend:8080/api/auth/login;
}

location ~ ^/api/storefront/checkout/ {
    limit_req zone=checkout_zone burst=5 nodelay;
    proxy_pass http://backend:8080$request_uri;
}

location /api/ {
    limit_req zone=api_zone burst=40 nodelay;
    proxy_pass http://backend:8080/api/;
}

location /app/ {
    proxy_pass http://staff-pwa:80/app/;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
}

location /admin/ {
    proxy_pass http://admin-web:80/admin/;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
}

location /store/ {
    proxy_pass http://storefront-web:80/store/;
    proxy_http_version 1.1;
```

```

    proxy_set_header Host $host;
}

```

The same vhost emits the shared security headers — X-Frame-Options: DENY, X-Content-Type-Options: nosniff, a Permissions-Policy that allows camera only on the same origin (needed by the Staff PWA), and a CSP pinning script-src and frame-src to the origin plus js.stripe.com.

2.2 Stateless JWT in an HttpOnly Cookie

Authentication is built on three properties: it is stateless, the access token is never exposed to JavaScript, and previously-issued tokens can be revoked instantly without a server-side session store. The implementation signs an HS256 JWT server-side and writes it into an HttpOnly, SameSite=Strict cookie called SHOE_AUTH; every token also carries a per-user token_version ("tv") claim. Bumping users.token_version in one UPDATE — on logout, password change or staff-disable — revokes every outstanding token for that user without a Redis blacklist.

```

// backend/.../security/JwtTokenProvider.java
public String generateToken(User user) {
    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + jwtProperties.getExpiration());

    return Jwts.builder()
        .subject(user.getUsername())
        .id(UUID.randomUUID().toString())
        .issuer(jwtProperties.getIssuer())
        .audience().add(jwtProperties.getAudience()).and()
        .issuedAt(now)
        .expiration(expiryDate)
        .claim("tv", user.getTokenVersion() == null ? 0L : user.getTokenVersion())
        .claim("uid", user.getId())
        .signWith(signingKey)
        .compact();
}

// backend/.../security/JwtAuthenticationFilter.java
String username = tokenProvider.getUsernameFromToken(jwt);
Long tokenVersion = tokenProvider.getTokenVersion(jwt);

User user = userRepository.findByUsername(username).orElse(null);
if (user == null) {
    log.debug("JWT for unknown user {}", username);
} else if (!user.isEnabled()) {
    log.debug("JWT for disabled user {}", username);
} else if (tokenVersion == null
    || (user.getTokenVersion() != null
    && tokenVersion < user.getTokenVersion())) {
    log.debug("JWT for user {} is revoked (tv={}, current={})",
        username, tokenVersion, user.getTokenVersion());
} else {
    UsernamePasswordAuthenticationToken authentication =
        new UsernamePasswordAuthenticationToken(

```

```

        user,
        null,
        user.getAuthorities()
    );
    SecurityContextHolder.getContext().setAuthentication(authentication);
}

```

Role-based access is enforced at the URL-pattern level inside SecurityConfig; `/api/storefront/**` is whitelisted as public. On the React side every Axios instance is created with `withCredentials: true` so the cookie rides along automatically and frontend code never touches the token.

2.3 Stripe Checkout with Stock Reservation

The checkout pipeline guarantees a unit of stock is consumed at most once. The flow runs in a single `@Transactional` method: every `(product_id, size)` row is read under a pessimistic JPA write lock, the stock is decremented and a `stock_movements` audit row is written atomically, and only after the reservation is committed does the backend create a Stripe `PaymentIntent`. Both the browser `/checkout/confirm` round-trip and the signed Stripe webhook converge on `applyPaymentIntentState`, the single private method allowed to flip the status to `PAID` — making the operation idempotent.

```

// backend/.../repository/ProductSizeRepository.java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT ps FROM ProductSize ps WHERE ps.product.id = :productId AND ps.size = :size")
Optional<ProductSize> findByProductIdAndSizeForUpdate(@Param("productId") Long productId,
                                                       @Param("size") BigDecimal size);

// backend/.../service/OrderService.java
private void applyPaymentIntentState(CustomerOrder order, String stripeStatus) {
    order.setStripePaymentStatus(stripeStatus);
    if ("succeeded".equals(stripeStatus)) {
        if (order.getStatus() == OrderStatus.PENDING) {
            order.setStatus(OrderStatus.PAID);
            order.setPaidAt(LocalDateTime.now());
            String emailToken = orderLookupTokenService.issue(
                order.getId(), order.getStripePaymentIntentId());
            orderEmailService.sendOrderConfirmation(order, emailToken);
        } else if (order.getStatus() == OrderStatus.CANCELLED) {
            log.warn("Stripe reports succeeded for cancelled order {} — requires manual review",
                order.getId());
        }
    } else if ("canceled".equals(stripeStatus) || "payment_failed".equals(stripeStatus)) {
        if (order.getStatus() == OrderStatus.PENDING) {
            restoreStock(order, "Online order (" + stripeStatus + ")");
            order.setStatus(OrderStatus.CANCELLED);
        }
    }
}
}

```

Because the storefront is anonymous, customers reach their own orders through a stateless HMAC-SHA256 token issued by OrderLookupTokenService at checkout (binds orderId, paymentIntentId and expMillis; verified in constant time). The public-facing identifier is an 8-character Crockford-base32 code with an STP- prefix, produced by OrderNumberService.

2.4 QR-Code Inventory and the Stock-Movements Audit Log

Each Product row carries an immutable qr_code_value UUID populated by PostgreSQL's gen_random_uuid() default. The Staff PWA scans the code with html5-qrcode against the rear camera; the Sell, Receive and Return actions reach the shared backend through three thin endpoints, and every action writes one immutable row to stock_movements. The same table is the authoritative audit trail and the data source the Admin Dashboard aggregates for revenue and units-sold charts, so the dashboard cannot diverge from the books.

```
// staff-pwa/src/components/QRScanner.jsx
const config = {
  fps: 10,
  qrbox: { width: 250, height: 250 },
  aspectRatio: 1.0,
};

await html5QrCodeRef.current.start(
  { facingMode: 'environment' },
  config,
  (decodedText) => {
    handleScanSuccess(decodedText);
  },
  (errorMessage) => {
    // Ignore scan errors (no QR code in frame)
  }
);

// backend/.../controller/ProductController.java
@PostMapping("/qr/{qrCode}/sell")
public ResponseEntity<ProductDTO> sellProductByQr(
    @PathVariable UUID qrCode,
    @RequestParam BigDecimal size,
    @RequestParam(defaultValue = "1") Integer quantity) {
    return ResponseEntity.ok(productService.sellProductByQrCode(qrCode, size, quantity));
}

-- backend/src/main/resources/db/migration/V7__create_stock_movements_table.sql
CREATE TABLE stock_movements (
  id BIGSERIAL PRIMARY KEY,
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  size DECIMAL(4, 1) NOT NULL,
  quantity INTEGER NOT NULL,
  direction VARCHAR(10) NOT NULL CHECK (direction IN ('IN', 'OUT')),
  reason VARCHAR(20) NOT NULL CHECK (reason IN ('SALE', 'RECEIPT', 'RETURN',
'ADJUSTMENT')),
```



```

    note VARCHAR(255),
    occurred_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_stock_movements_product ON stock_movements(product_id);
CREATE INDEX idx_stock_movements_user ON stock_movements(user_id);
CREATE INDEX idx_stock_movements_date ON stock_movements(occurred_at);

```

AnalyticsService reads stock_movements with DAY / MONTH / YEAR group-by modes, pushing the aggregation down to the database in one round trip. The migrations create two purpose-built indexes used by this read path — idx_stock_movements_date for the time-window scan and idx_stock_movements_product for per-product rollups — so the dashboard's scan cost is proportional to the time window rather than the total table size.

2.5 AI-Assisted Product Photography and Schema Evolution

The Admin Panel includes an AI image-generation panel for shops without a dedicated photographer. The owner picks a shooting mode, a city for the background, a time of day and a focus area; the server calls Google Gemini 2.5 Flash Image with three prompt variants drawn from a per-city landmark pool, and the owner promotes one of the previews to the storefront image. The endpoint is admin-only at the controller level, uploaded mime types are validated server-side, and the AI provider's API key never leaves the backend container.

```

// backend/.../controller/AdminProductAiController.java
@RestController
@RequestMapping("/api/admin/products")
@RequiredArgsConstructor
@PreAuthorize("hasRole('ADMIN')")
public class AdminProductAiController {

    private final ProductImageGenerationService productImageGenerationService;

    @PostMapping("/generate-image")
    public ResponseEntity<GenerateProductImageResponse> generateImage(
        @Valid @RequestBody GenerateProductImageRequest request) {
        return ResponseEntity.ok(productImageGenerationService.generateImage(request));
    }
}

```

Schema evolution is managed by Flyway: twenty ordered migrations (V1–V20) cover categories, products, the per-size stock split (V6), stock_movements (V7), customer orders, the Stripe + shipping extension (V12), the per-user token_version column that enables JWT revocation (V17), the admin audit log (V18), and the public order_number column (V20). Migrations run on container boot, so the database is reproducible from an empty PostgreSQL volume.

3. Screenshots and Visualisations

3.1 Admin Panel

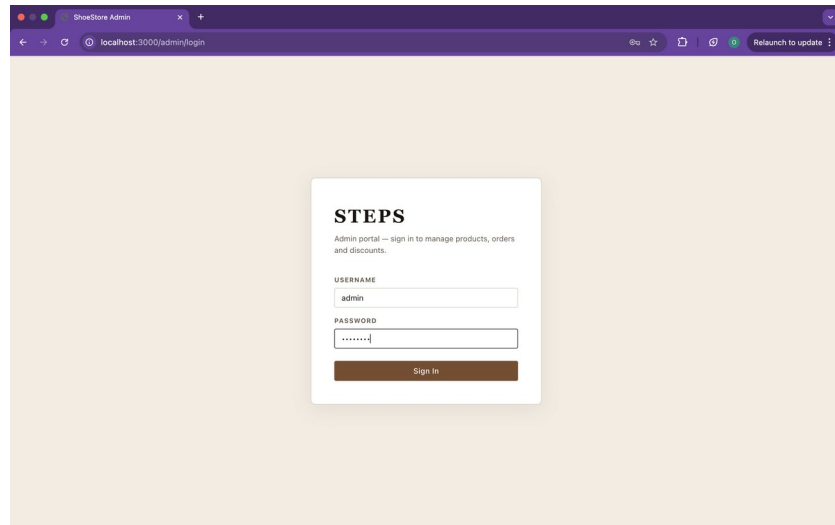


Figure 1. Admin login (/admin/login).

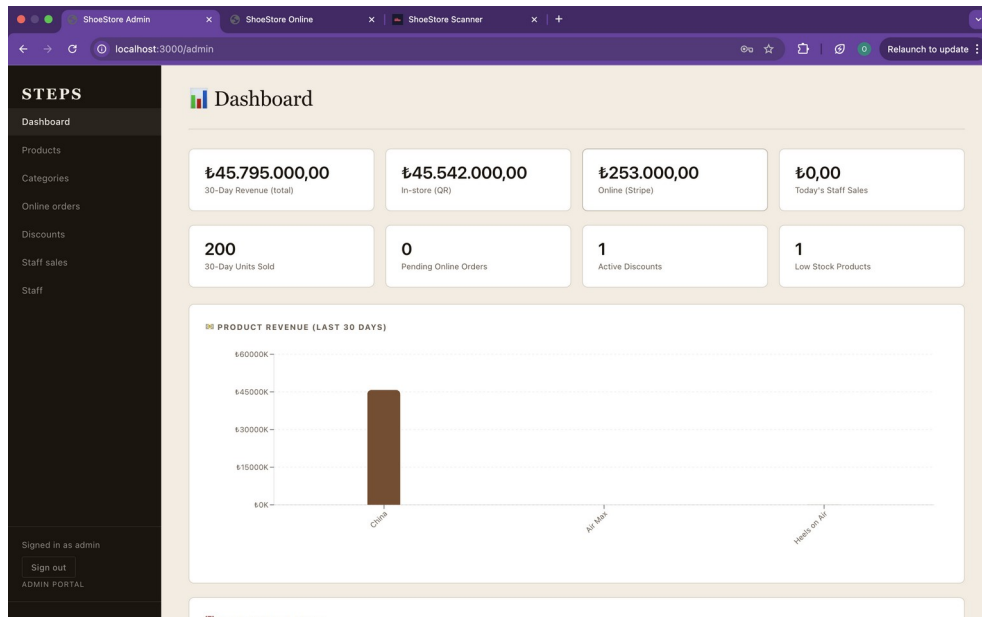


Figure 2. Admin dashboard — 30-day revenue, units sold, pending orders, per-product revenue chart, served by AnalyticsService over stock_movements.

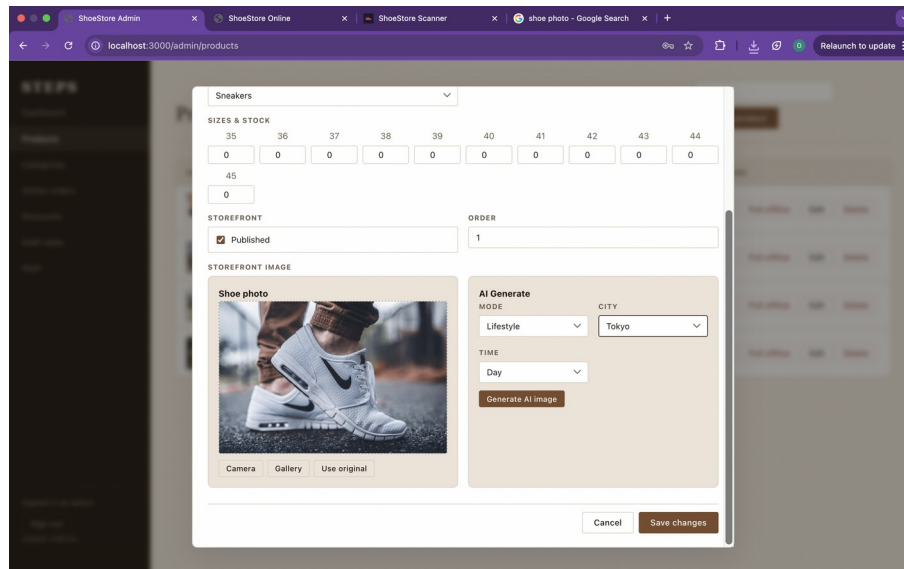


Figure 3. Product editor — per-size stock and the AI image-generation panel (mode, city, time-of-day).

ID	CUSTOMER	CONTACT	ITEMS	TOTAL	PAYMENT	STATUS	ACTIONS
#5	Oğuzhan Türksoy	oguzhanturks06@gmail.com 5372802405	1	€1.000,00	SUCCEEDED	PAID	PAID
#4	Oğuzhan Türksoy	oguzhanturks06@gmail.com 5372802405	1	€1.000,00	SUCCEEDED	PAID	PAID
#3	Oğuzhan Türksoy	oguzhanturks06@gmail.com 5372802405	1	€1.000,00	SUCCEEDED	PAID	PAID
#2	Oğuzhan Türksoy	oguzhanturks06@gmail.com 5372802405	1	€250.000,00	SUCCEEDED	PAID	PAID

Figure 4. Online orders — Stripe-paid orders with the PAID → FULFILLED status control.

3.2 Customer Storefront

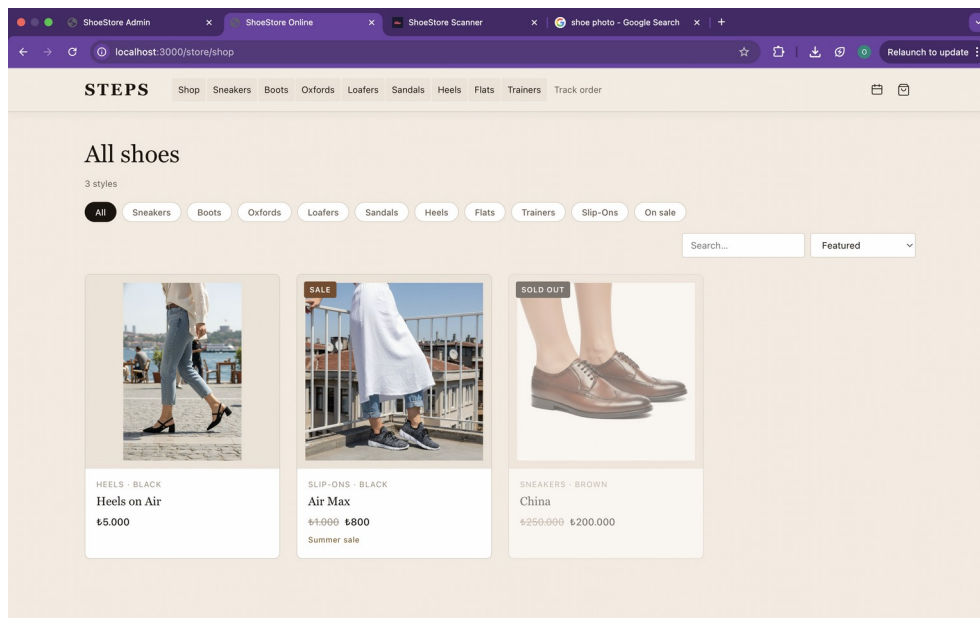


Figure 5. Storefront catalogue with category filter and SALE / SOLD OUT badges.

The screenshot shows the 'Checkout' page of the STEPS storefront. The page is divided into three main sections: 'Contact', 'Shipping', and 'Payment'. The 'Contact' section includes fields for 'Full name' (Oguzhan Turker), 'Email' (oguzhanurkucuk@gmail.com), and 'Phone'. The 'Shipping' section includes fields for 'Address line 1' (Sardislar Mahallesi 101), 'Address line 2 (optional)' (14), 'City' (Beyoglu / Beşiktaş), 'Postal code' (34712), and 'Country' (TR). The 'Payment' section includes a 'Card number' (4242 4242 4242 4242), 'Expiration (MM/YY)' (11/28), 'Security code' (888), 'Country' (Poland), and a 'Full name' field. There is also a 'Save my information for faster checkout' checkbox and a 'Pay now' button. An 'Order summary' box on the right shows 'Heels on Air' for \$5,000.00 and a 'Total' of \$5,000.00.

Figure 6. Checkout page — contact, shipping, embedded Stripe Elements card form.

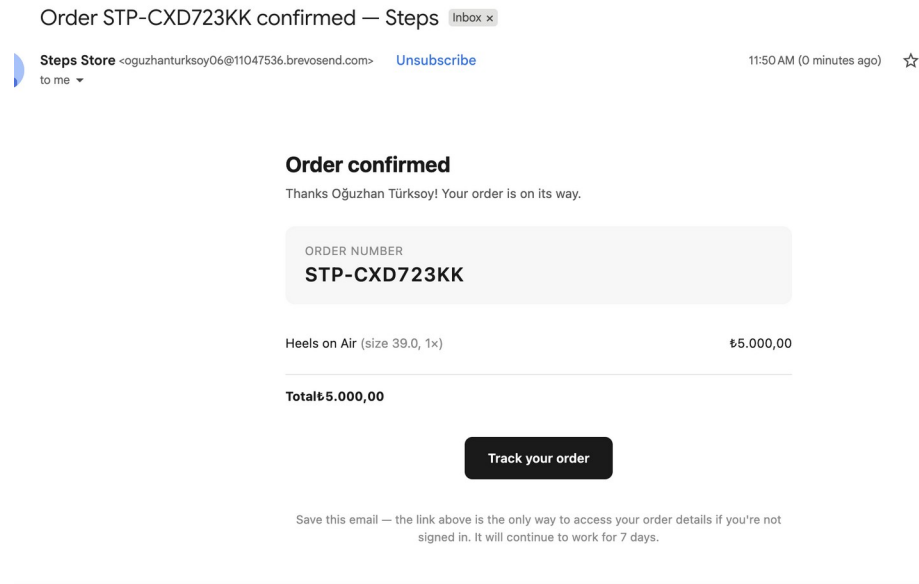


Figure 7. Order-confirmation email (public STP-XXXXXXXX number + HMAC-signed Track Order link).

3.3 Staff PWA

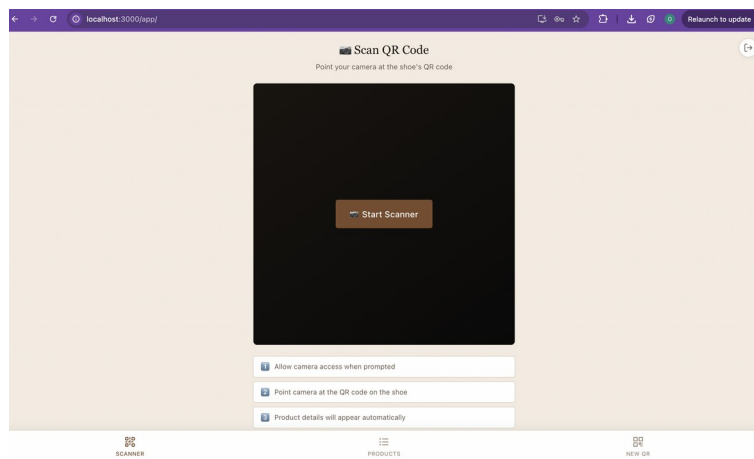


Figure 8. Staff PWA scanner — `html5-qrcode` viewport (`facingMode: 'environment'`).

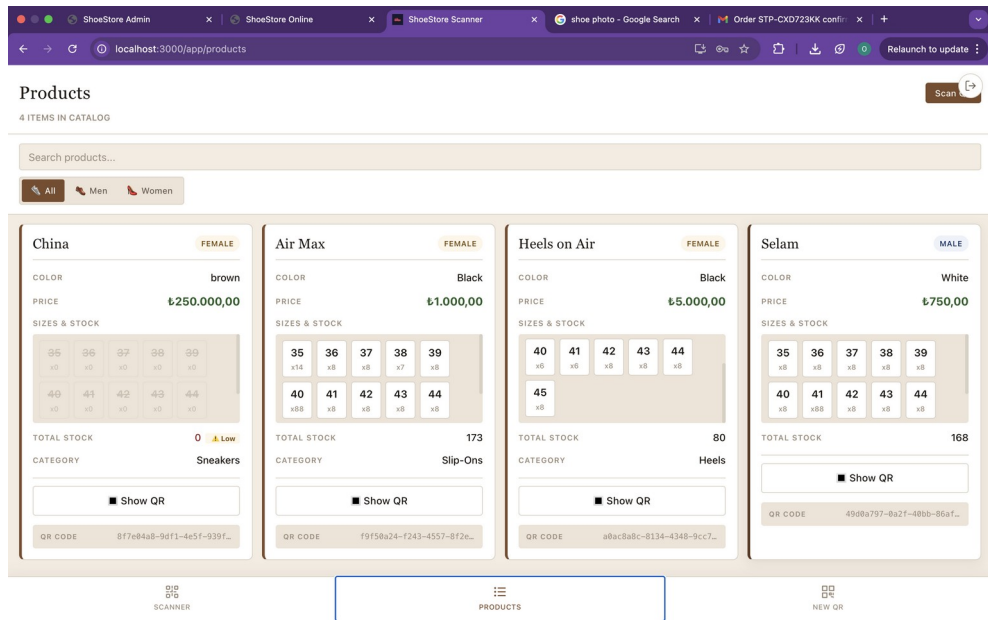


Figure 9. Staff PWA products list — per-size stock, price, inline QR code.

4. Conclusions and Further Development Perspectives

The ShoeStore project achieves its stated goal: a single deployable platform that runs both the physical shoe store and its online channel on top of one Spring Boot backend and one PostgreSQL database. All three target use cases — owner management, in-store staff operations and anonymous customer checkout — are reachable from one URL behind one nginx proxy, and ``docker compose up --build`` brings the entire stack online deterministically.

The shared-backend architecture proved to be the correct decision for the requirements. Because all three frontends mutate the same PostgreSQL rows, inventory consistency is enforced by the schema (per-size row, pessimistic write lock at checkout) rather than by a synchronisation job — and the `stock_movements` audit table doubles as the data source for the revenue dashboard, so the books and the charts cannot disagree by construction. The Stripe `PaymentIntent` + idempotent webhook design has not lost or duplicated an order across the full development cycle.

The resulting codebase is concretely measurable. The backend is 6,565 lines of Java spread across 12 controllers, 19 services, 14 JPA entities and 12 repositories exposing 41 distinct REST endpoints; the database schema is owned by 20 Flyway migrations (15 tables and 34 indexes); the three React frontends together amount to 6,306 lines of JSX/JS across 35 components and pages. These numbers were obtained by static counting on the same source tree the code snippets in section 2 were extracted from.

Several directions are worth exploring next. Customer accounts with order history and saved shipping addresses are the most obvious addition; the `order_number` / lookup-token pair already lets the same anonymous order be claimed retroactively, so the migration path is straightforward. The analytics surface can grow beyond the 30-day snapshot — cohort retention, conversion funnel, and per-staff performance reports would all reuse the existing `stock_movements` table without additional schema. Finally, the AI image-generation service could be extended to render full lookbooks from a single shoe photo, and a small mobile-first storefront PWA would let customers install the shop on their phone the same way staff already install the Staff PWA.

5. References

- [1] Spring Boot 3 Reference Documentation. Broadcom, 2025.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] Spring Security Reference — Authentication and Authorization. Broadcom, 2025. <https://docs.spring.io/spring-security/reference/>
- [3] Spring Data JPA Reference. Broadcom, 2025. <https://docs.spring.io/spring-data/jpa/reference/>
- [4] Flyway Documentation — Versioned Database Migrations. Redgate Software, 2025. <https://documentation.red-gate.com/fd/>
- [5] PostgreSQL 15 Documentation — Concurrency Control and Row-Level Locks. The PostgreSQL Global Development Group, 2025. <https://www.postgresql.org/docs/15/>
- [6] React 18 Documentation. Meta Platforms, 2025. <https://react.dev/>
- [7] Vite Build Tool Guide. Evan You et al., 2025. <https://vitejs.dev/guide/>
- [8] Stripe PaymentIntents API Reference. Stripe, Inc., 2025. <https://stripe.com/docs/payments/paymentintents>
- [9] Stripe Webhooks — Signature Verification. Stripe, Inc., 2025. <https://stripe.com/docs/webhooks/signatures>
- [10] Stripe Elements — Web SDK Documentation. Stripe, Inc., 2025. <https://stripe.com/docs/stripe-js>
- [11] JWT — Java JWT Library. jwtk/jjwt, 2025. <https://github.com/jwtk/jjwt>
- [12] html5-qrcode — Browser QR-Code Scanning Library. mebjas, 2025. <https://github.com/mebjas/html5-qrcode>
- [13] ZXing — Multi-Format 1D/2D Barcode Image Processing for Java. zxing/zxing, 2025.
<https://github.com/zxing/zxing>
- [14] nginx Documentation — Reverse Proxying, Rate Limits, Security Headers. F5, Inc., 2025.
<https://nginx.org/en/docs/>
- [15] Docker Compose v2 Reference. Docker, Inc., 2025. <https://docs.docker.com/compose/>
- [16] Progressive Web Apps. MDN Web Docs, Mozilla Foundation, 2025.
https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- [17] Google Gemini API — Image Generation. Google LLC, 2025. <https://ai.google.dev/gemini-api/docs>
- [18] OWASP — JSON Web Token Cheat Sheet. The OWASP Foundation, 2025.
https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html